

DMonitor Java SDK集成开发指南

v1.0.0

文档说明

本指南用于说明 dmonitor-java-lib 的集成方式、Java 公共接口、JNA 原生结构映射、输入输出模型、状态码和故障排查方法。文档内容依据当前项目源码、构建配置及运行验证结果整理。

本项目封装的 libDMonitor 原生算法库及其 license 授权范围，以实际交付协议和原生库许可为准。项目中使用的第三方组件遵循各自的开源许可。

文档基线：Git 提交 3b8136f；Java SDK 版本 1.0.0；验证环境原生算法版本 2.0.6.8。不同平台交付的原生库版本可能不同，应以 MonitorService.getVersion() 的运行时返回值为准。

免责声明

本指南按当前源码的实际行为编写，不构成对特定业务适用性、结果精度或持续兼容性的额外保证。调用方应自行核验 RTCM3 数据质量、星历覆盖范围、license 有效性、时间窗口及原生库平台匹配关系。

文档后续可随 SDK 演进更新。接口行为发生变更时，应同时更新版本记录、示例和兼容性说明。

文档修改记录

版本	修改日期	描述
v1.0.0	2026-07-29	初版发布：依据当前工程源码生成完整集成、API、模型和故障排查指南

一、SDK简介

DMonitor Java SDK 是 libDMonitor 原生算法库的 Java/JNA 封装，用于接收监测站与基准站的 GNSS RTCM3 数据，执行监测解算，并将原生文本结果转换为结构化 MonitorData。

SDK 提供两种调用层级：在 Spring Boot 应用中推荐注入 MonitorService；非 Spring 场景可直接使用 MonitorLibrary.INSTANCE 调用原生接口。

层级	入口	适用场景	主要职责
业务服务层	MonitorService	Spring Boot 应用，推荐	校验输入、映射原生结构、调用算法、解析结果并回调
结果解析层	MonitorDataService	自定义结果处理或同步解析	将原生结果文本转换为 MonitorData
原生绑定层	MonitorLibrary	非 Spring 或高级集成	通过 JNA 直接加载并调用 libDMonitor

二、名词解释

名词	说明
GNSS	全球卫星导航系统的统称。
RTCM3	监测站和基准站输入使用的二进制差分数据格式。
BRDC / RINEX	广播星历数据。当前 Java 包允许不传，传入时会自动补 0 结尾。
Rover	监测站或流动站，映射到 roverBytes、roverName。
Base	基准站，映射到 baseBytes、baseName。
JNA	Java Native Access，用于将 Java 调用映射到 libDMonitor 的 C ABI。
ECEF	地心地固坐标系。baseX、baseY、baseZ 对应基准站 ECEF 坐标。
E / N / U	解算结果的东、北、天方向分量。
Fixed Rate	固定解比例，对应 minFixedRate 输入和 fixedRate 输出。
navsys	传给原生库的卫星系统聚合值，由 navSys 中的整数项求和得到。

三、快速开始

1. 准备工作

集成前应准备与运行平台匹配的原生库、有效 license、监测站 RTCM3 数据和基准站 RTCM3 数据。广播星历 BRDC 数据可选。

项目	要求
JDK	20 或以上；工程编译 release 为 20，验证环境使用 Java 21。
Spring Boot	3.2.x；当前 pom.xml 使用 3.2.4。
JNA	5.12.1；由工程依赖提供。
Maven	使用项目自带 Maven Wrapper，或兼容的 Maven 3.x。
license	调用 startMonitor/startGMonitor 时提供有效文件路径。
输入数据	roverBytes 与 baseBytes 必须非空；brdcBytes 可空。

2. 支持平台

操作系统	架构	工程内置资源	状态
Linux	x86-64	linux-x86-64/libDMonitor.so	已内置
Linux	AArch64	linux-aarch64/libDMonitor.so	已内置
macOS	Apple Silicon / arm64	darwin-aarch64/libDMonitor.dylib	已内置
Windows	未提供	无	当前工程不支持
macOS	Intel x86-64	无	当前工程不支持

注意：JNA 通过 `Native.load("DMonitor")` 加载原生库。若打包方式或类路径导致自动提取失败，可用 `-Djna.library.path=/path/to/native/lib/dir` 显式指定目录。

3. 安装 SDK

当前工程的 Maven 坐标为 `com.navfirst:dmonitor-java-lib:1.0.0`。若尚未发布到团队仓库，可先在本工程执行本地安装。

```
./mvnw clean install
```

在调用方项目中加入依赖：

```
<dependency>
  <groupId>com.navfirst</groupId>
  <artifactId>dmonitor-java-lib</artifactId>
  <version>1.0.0</version>
</dependency>
```

Spring Boot 3 应用会通过 `AutoConfiguration.imports` 自动加载 `RtkconvAutoConfiguration`，并注册 `MonitorService` 和 `MonitorDataService`。调用方也可以声明同类型 Bean 覆盖默认实现。

4. 接入流程

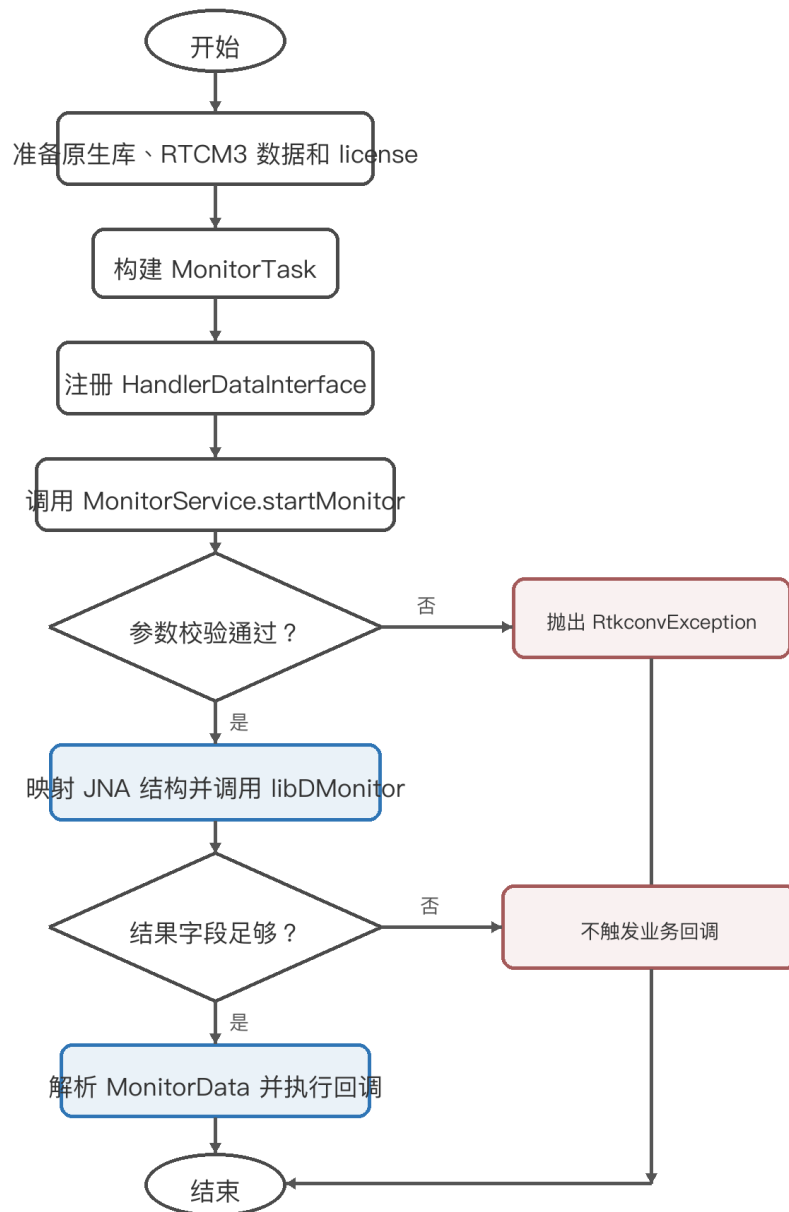


图 1 MonitorService 推荐接入流程

5. 获取算法版本和机器标识

```
String nativeVersion = monitorService.getVersion();
String machineUuid = monitorService.getUuid();
```

getVersion() 返回当前平台实际加载的原生算法版本。验证环境返回 2.0.6.8。getUuid() 返回机器唯一标识，通常用于申请与当前设备绑定的 license。

安全： 机器 UUID 属于设备标识信息。日志、截图和工单中应按项目安全要求处理，不要写入公共代码仓库。

6. 构建任务并启动解算

```
import com.navfirst.dmonitor.lib.domains.MonitorTask;
import com.navfirst.dmonitor.lib.services.MonitorService;

import java.nio.file.Files;
import java.nio.file.Path;

Path brdcPath = Path.of("/data/BRDM1400.rnx");
byte[] brdc = Files.isRegularFile(brdcPath)
    ? Files.readAllBytes(brdcPath)
    : null;

MonitorTask task = MonitorTask.builder()
    .rtMode(3)
    .timeStart("2026/05/20 00:00:00")
    .timeEnd("2026/05/20 01:00:00")
    .sample(0)
    .vrs(0)
    .roverName("XPJZ01")
    .baseName("XPJZ02")
    .brdcBytes(brdc)
    .roverBytes(Files.readAllBytes(Path.of("/data/rover.rtc3")))
    .baseBytes(Files.readAllBytes(Path.of("/data/base.rtc3")))
    .outMode(1)
    .minFixedRate(0.75)
    .navSys("1,4,8,32")
    .bds(1)
    .build();

monitorService.setHandlerData(result -> {
    System.out.println("解算结果: " + result);
});

monitorService.startMonitor(task, "/data/license.lic");
```

注意：当前 startMonitor() 是同步调用：它在调用线程上执行原生算法，解析完成后再调用已注册的 HandlerDataInterface。耗时任务应由调用方放到受控的后台线程或任务执行器中。

四、详细接口描述

注意：本章只描述当前工程实际公开的 Java 接口。方法签名与默认行为以源码为准。

1. 获取原生算法版本

返回当前进程实际加载的 libDMonitor 版本，用于部署核验和问题定位。

1.1 接口定义

```
String MonitorService.getVersion();
```

1.2 参数说明

无。

1.3 返回值说明

返回类型	说明
String	原生算法版本字符串；验证环境返回 2.0.6.8。

2. 获取机器唯一标识

获取当前机器标识，通常用于生成或申请设备绑定 license。

2.1 接口定义

```
String MonitorService.getUuid();
```

2.2 参数说明

无。

2.3 返回值说明

返回类型	说明
String	当前设备唯一标识。具体格式由原生库决定。

3. 注册结果处理器

设置解算结果回调。MonitorService 将其转交给 MonitorDataService 保存。

3.1 接口定义

```
void MonitorService.setHandlerData(
    HandlerDataInterface handlerDataInterface);

@FunctionalInterface // 语义上可用 Lambda，源码未显式标注
public interface HandlerDataInterface {
    void handlerData(MonitorData monitorData);
}
```

3.2 参数说明

参数名	必填	参数类型	说明
handlerDataInterface	建议是	HandlerDataInterface	结果回调；可使用 Lambda。 传 null 会清除当前处理器，之后不再回调。

3.3 线程与实例行为

- 默认 MonitorDataService 是 Spring 单例，内部只保存一个 handlerDataInterface；后一次注册会覆盖前一次。
- 回调在 startMonitor() 的调用线程中执行，SDK 未创建专用回调线程。
- 回调异常不会被 SDK 捕获，会继续向调用方传播。

4. 启动监测解算

校验输入数据、构建 JNA 结构、调用 libDMonitor，并将原生返回结果转换为 MonitorData 后执行回调。

4.1 接口定义

```
void MonitorService.startMonitor(MonitorTask monitorTask, String license);
```

4.2 参数说明

参数名	必填	参数类型	说明
monitorTask	是	MonitorTask	监测任务。对象本身、roverBytes、baseBytes 会在 Java 层校验。
license	业务必填	String	license 文件路径。Java 层不校验空值或文件存在性，结果由原生库决定。

4.3 返回值说明

无直接返回值。原生结果通过 HandlerDataInterface 回调。未注册回调时，算法仍会执行，但结构化结果不会交给业务方。

4.4 异常与边界

场景	行为
monitorTask 为 null	抛出 RtkconvException，消息为“监测任务不能为空”。
roverBytes 为 null 或空	抛出 RtkconvException，消息为“监测站数据流不能为空”。
baseBytes 为 null 或空	抛出 RtkconvException，消息为“基准站数据流不能为空”。
时间格式不支持	TimeUtils.parseEpoch() 抛出 IllegalArgumentException。
navSys 含非整数项	整数解析失败并抛出 NumberFormatException。
原生返回空数据或字段少于 23 项	MonitorDataService 不触发回调。
roverName/baseName 为 null 且结果可解析	结果构建阶段调用 trim()，可能抛出 NullPointerException。

5. 同步解析原生结果

高级场景可直接使用 MonitorDataService 将原生文本转换为 MonitorData，而不触发回调。

5.1 接口定义

```
MonitorData MonitorDataService.handlerDataSync(  
    MonitorTask monitorTask,  
    String data,  
    String errMsg);
```

5.2 参数说明

参数名	必填	参数类型	说明
monitorTask	是	MonitorTask	为结果补充站名、任务模式、卫星系统等上下文。
data	是	String	原生结果第一行；空白或字段不足时返回 null。
errMsg	否	String	原生结果第二行；非空时写入 MonitorData.errMsg。

5.3 返回值说明

返回类型	说明
MonitorData	解析成功时返回结构化结果；data 为空或少于 23 个非空字段时返回 null。

6. 直接调用原生监测接口

非 Spring 场景可以直接调用 MonitorLibrary。调用方需要自行构造 MonitorStreamInfo、维护结构字段规则并解析两行返回文本。

6.1 接口定义

```
String MonitorLibrary.startGMonitor(MonitorStreamInfo monitorInfo, String license);
```

6.2 参数说明

参数名	必填	参数类型	说明
monitorInfo	是	MonitorStreamInfo	与原生 GMonitorStreamInfo ABI 对齐的 JNA 结构。
license	是	String	license 文件路径。

6.3 返回值说明

返回两行文本：第一行为解算数据 solBuf，第二行为错误信息 errMsg。Java 服务层使用 split(\\R, 2) 拆分，因此第二行之后的剩余内容会整体归入 errMsg。

```
<solBuf>
<errMsg>
```

注意：MonitorLibrary 同时声明 INSTANCE 和 INSTANCES。INSTANCES 是 JNA synchronizedLibrary 包装，但默认 MonitorServiceImpl 使用的是 INSTANCE，因此 SDK 服务层不会自动串行化并发原生调用。

7. 原生结果字段映射

MonitorDataService 先按空格拆分并过滤空项，再依据字段数解析。当前 24 项格式在第 17 项保留一个未映射字段；23 项格式不包含该保留项。

序号	原生字段	MonitorData 字段	说明
1	endDate	不映射	原生结果结束日期或首段日期。
2	endTime	不映射	原生结果结束时间或首段时间。
3	lastGpsDate	gpsTime / lastObsTime	与第 4 项拼接。
4	lastGpsTime	gpsTime / lastObsTime	与第 3 项拼接。
5	dposMax	dposmax	位置差最大值。
6	dposAvg	dposavg	位置差平均值。
7	dposStd	dposstd	位置差标准差。
8	fixedRate	fixedRate	固定率。
9	roverEpochRate	roverEpochRate	监测站历元完整率。

序号	原生字段	MonitorData 字段	说明
10	baseEpochRate	baseEpochRate	基准站历元完整率。
11	E	E	东向分量。
12	N	N	北向分量。
13	U	U	天向分量。
14	solStatus	solStatus	解算状态文本。
15	solutionType	solutionType	解算质量类型。
16	satNum	satNum	卫星数量。
17	reserved	不映射	仅 24 项布局存在；当前实现跳过。
18	roverSample	roverSample	监测站采样率。
19	baseSample	baseSample	基准站采样率。
20	roverObsNum	roverObsNum	监测站历元数量。
21	baseObsNum	baseObsNum	基准站历元数量。
22	fileStatus	fileStatus	观测文件状态码。
23	navStatus	navStatus	星历状态码。
24	offTime	offTime	解算耗时。

注意：23 项格式中不存在第 17 项 reserved，roverSample 到 offTime 依次前移一位。SDK 还保留对超过 24 项的旧版紧凑格式解析，但业务方不应依赖该内部兼容分支。

五、数据模型

1. MonitorTask 输入模型

MonitorTask 是推荐服务层的统一输入。下表将“SDK校验”和“业务建议”区分开，便于识别当前代码实际强制项。

字段	类型	默认值	要求	说明
rtMode	int	3	建议	解算模式：0 为 SPP，2 为动态，3 为静态。
timeStart	String	null	建议	开始时间；支持 yyyy/MM/dd HH:mm:ss 和 yyyy-MM-dd HH:mm:ss。空白时传全 0 时间。
timeEnd	String	null	建议	结束时间；格式同 timeStart。空白时传全 0 时间。
filterPeriod	Integer	null	否	滤波时长，单位秒；不传原生库，仅复制到结果。
processInterval	Integer	null	否	任务间隔，单位秒；不传原生库，仅复制到结果。

字段	类型	默认值	要求	说明
sample	Integer	null -> 0	否	采样间隔；0 表示由算法估算。
vrs	int	0	否	VRS 模式：0 否，1 是。
roverName	String	null	业务必填	监测站名称；结果解析时会调用 trim()。
baseName	String	null	业务必填	基准站名称；结果解析时会调用 trim()。
brdcBytes	byte[]	null	否	广播星历；非空时服务层自动补 0 终止字节。
baseBytes	byte[]	null	是	基准站 RTCM3 数据；Java 层校验非空。
roverBytes	byte[]	null	是	监测站 RTCM3 数据；Java 层校验非空。
outMode	int	1	否	输出模式：0 为历元解，1 为单一解。
outFile	String	null -> 空串	否	底层算法输出文件路径。
minFixedRate	Double	null -> 0	否	最小固定率阈值，例如 0.75。
taskType	int	0	否	任务类型：0 准实时，1 重算；仅复制到结果。
navSys	String	空白 -> 13	否	逗号分隔整数，服务层求和后写入 navsys。常用示例为 1,4,8,32。
bds	int	0	否	源码注释：0 为 B1I+B3I，1 为 B1C+B2A；当前实现还会在 0 且 navSys 含 32 时减去 32。
extra	String	null	否	业务扩展信息；仅复制到结果。
baseX	Double	null -> 0	否	基准站 ECEF X 坐标。
baseY	Double	null -> 0	否	基准站 ECEF Y 坐标。
baseZ	Double	null -> 0	否	基准站 ECEF Z 坐标。

注意：当 navSys 为空时，传给原生库的 navsys 默认值为 13；但结果对象 navSys 会显示默认字符串 1,4,8,32。两者并不等价，生产调用建议显式传 navSys。

2. MonitorData 输出模型

字段	类型	说明
roverName	String	监测站名称，来自 MonitorTask。
baseName	String	基准站名称，来自 MonitorTask。
gpsTime	String	解算坐标时间，由原生结果第 3、4 项拼接。
lastObsTime	String	最后历元时间；当前实现与 gpsTime 相同。
E	Double	东向坐标或位移分量。
N	Double	北向坐标或位移分量。
U	Double	天向坐标或位移分量。
solStatus	String	原生解算状态文本。
satNum	Integer	参与解算的卫星数量。
roverSample	Integer	监测站采样率。
baseSample	Integer	基准站采样率。
fileStatus	int	观测文件状态码。
fileStatusDesc	String	由 ObsErrorEnum 转换的中文说明。
navStatus	int	星历状态码。
navStatusDesc	String	由 NavErrorEnum 转换的中文说明；未知值返回 null。
roverObsNum	int	监测站历元数量。
baseObsNum	int	基准站历元数量。
offTime	String	解算耗时文本。
rtMode	int	来自 MonitorTask。
vrs	int	来自 MonitorTask。
taskType	int	来自 MonitorTask。
filterPeriod	Integer	来自 MonitorTask，单位秒。
processInterval	Integer	来自 MonitorTask，单位秒。
fixedRate	Double	原生结果固定率。
dposmax	Double	位置差最大值。
dposavg	Double	位置差平均值。
dposstd	Double	位置差标准差。
outMode	int	来自 MonitorTask。
errMsg	String	原生返回第二行，非空时写入。
roverEpochRate	Double	监测站历元完整率。
baseEpochRate	Double	基准站历元完整率。
solutionType	int	解算质量类型。
navSys	String	任务卫星系统字符串；空白时显示 1,4,8,32。
extra	String	任务扩展信息。

3. MonitorStreamInfo 原生结构映射

MonitorStreamInfo 继承 JNA Structure，对应原生 GMonitorStreamInfo。字段顺序属于 ABI 合约，不得调整。

```
typedef struct GMonitorStreamInfo {
    int32_t calMode;
    double es[6];
    double ee[6];
    int32_t ti;
    int32_t vrsMode;
    int32_t navsys;
    int32_t solstatic;
    double fixThresh;
    double rb[3];
    const char* outfile;
    const uint8_t* brdc_buf;
    uint64_t brdc_len;
    const uint8_t* rover_buf;
    uint64_t rover_len;
    const uint8_t* base_buf;
    uint64_t base_len;
} GMonitorStreamInfo;
```

Java 字段	原生字段	原生类型	说明
calMode	calMode	int32_t	解算模式。
es	es[6]	double[6]	开始时间：年、月、日、时、分、秒。
ee	ee[6]	double[6]	结束时间：年、月、日、时、分、秒。
ti	ti	int32_t	采样间隔。
vrsMode	vrsMode	int32_t	VRS 模式。
navsys	navsys	int32_t	卫星系统聚合值。
solstatic	solstatic	int32_t	输出模式。
fixThresh	fixThresh	double	最小固定率。
rb	rb[3]	double[3]	基准站 ECEF 坐标。
outfile	outfile	const char*	输出文件路径。
brdcBuf	brdc_buf	const uint8_t*	广播星历内存指针。
brdcLen	brdc_len	uint64_t	广播星历字节数。
roverBuf	rover_buf	const uint8_t*	监测站数据内存指针。
roverLen	rover_len	uint64_t	监测站数据字节数。
baseBuf	base_buf	const uint8_t*	基准站数据内存指针。
baseLen	base_len	uint64_t	基准站数据字节数。

注意：setBrdc/setRover/setBase 会把 byte[] 写入 JNA Memory，并由 MonitorStreamInfo 的 transient 字段持有该内存，避免原生调用期间被回收。直接调用时不要只设置长度而遗漏对应指针。

六、状态码

1. 观测数据状态 ObsErrorEnum

枚举项	值	说明
NORMAL	0	正常
ROVER_RTCM_NOT_UPDATE	1	监测站数据未更新
BASE_RTCM_NOT_UPDATE	2	基准站数据未更新
ALL_RTCM_NOT_UPDATE	3	基准站和监测站数据都未更新
ROVER_RTCM_NOT_ENOUGH	4	监测站数据不足
BASE_RTCM_NOT_ENOUGH	8	基准站数据不足
ALL_RTCM_NOT_ENOUGH	12	基准站和监测站数据都不足
ROVER_RTCM_ERROR_FORMAT	16	监测站无数据或数据格式不为 RTCM3
ROVER_NO_DATA	21	监测站无数据或数据格式不为 RTCM3
RTCM_ERROR_FORMAT	29	监测站无数据或基准站数据不足
BASE_RTCM_ERROR_FORMAT	32	基准站无数据或数据格式不为 RTCM3
BASE_NO_DATA	42	基准站无数据
NO_ALL_DATA	63	任何数据都没有
OTHER	999	基准站或监测站数据不足；未知 fileStatus 也映射到此说明

2. 星历状态 NavErrorEnum

枚举项	值	说明
NORMAL	0	正常
ROVER_RTCM_NOT_NAV	1	监测站无星历
BASE_RTCM_NOT_NAV	2	基准站无星历
NAV_NOT_ENOUGH	4	星历不足
NAV_NOT_ROVER	5	监测站无星历
NAV_NOT_BASE	6	基准站无星历
NAV_FILE_NOT_ENOUGH	8	设置的星历文件星历不足
NAV_ERROR	15	无任何星历

注意：NavErrorEnum 对未知 navStatus 返回 null；调用方展示状态时应保留原始数值，并为未知描述提供兜底文本。

七、使用限制说明

- 平台必须与工程内置原生库匹配。当前不包含 Windows 和 Intel macOS 原生库。
- startMonitor() 为同步阻塞调用，耗时取决于输入数据量和原生算法。不要在 Web 请求主线程或 UI 主线程直接执行长任务。

- 默认 MonitorServiceImpl 使用非同步包装的 MonitorLibrary.INSTANCE。并发调用是否安全取决于原生库，当前 Java 层未提供互斥保护。
- 默认结果处理器是单槽位可变状态。多个业务方共用同一 Spring 单例时，后注册的回调会覆盖先注册的回调。
- MonitorStreamInfo 字段顺序、数组长度和整数宽度属于原生 ABI 合约，不可随意修改。
- roverBytes 和 baseBytes 必须非空；brdcBytes 可以为空。直接 JNA 调用时，非空 BRDC 建议手动追加 0 终止字节。
- license 路径、时间范围、数据格式和站名完整性未被 Java 层完全校验，调用方应在进入 SDK 前完成业务校验。
- navSys 使用逗号分隔的整数项并直接求和，不接受空项、空格项或非整数文本。建议使用受控配置生成该字符串。
- 当前原生结果解析依赖空格分隔字段数。格式变化可能导致无回调或数值转换异常，升级原生库时必须执行兼容性回归。
- SDK 未提供显式 cleanup/close 接口。每次 startMonitor() 是一次独立调用，进程级原生资源生命周期由 JNA 与原生库管理。

八、常见问题

1. 启动时报 UnsatisfiedLinkError

- 确认运行系统与 CPU 架构在支持矩阵内。
- 确认 libDMonitor.so 或 libDMonitor.dylib 已进入最终应用 classpath。
- 必要时设置 -Djna.library.path=/absolute/path/to/native/lib。
- 检查原生库的间接动态依赖是否齐全，可使用 ldd 或 otool -L 排查。

2. 如何申请 license

- 在目标部署机器上调用 monitorService.getUuid() 获取机器标识。
- 按交付流程使用该标识申请 license，并将 license 文件部署到应用可读路径。
- 不要在其他机器生成 UUID 后复用设备绑定 license。

3. startMonitor() 执行后没有回调

- 确认已在调用前注册 HandlerDataInterface，且未被其他组件覆盖。
- 确认原生返回第一行非空，并至少包含 23 个空格分隔字段。
- 查看应用日志中的原生返回和 errMsg，核对 license、数据格式、时间范围和星历。
- 确认 roverName、baseName 非空，避免结果构建阶段异常。

4. 监测结果提示数据不足或格式错误

- 核对 roverBytes/baseBytes 是否为 RTCM3 数据，而非路径字符串或压缩包字节。
- 确认两个数据流覆盖相同或可解算的时间窗口。
- 根据 fileStatus 和 fileStatusDesc 定位监测站、基准站或双方的数据问题。

5. 星历状态异常

- 确认 BRDC/RINEX 文件日期与观测数据时间一致。
- 即使 brdcBytes 为空，RTCM3 流中也需要有足够星历信息。
- 结合 navStatus 原始值和 navStatusDesc 判断缺少监测站星历、基准站星历或星历文件覆盖。

6. 时间格式报错

- 只使用 yyyy/MM/dd HH:mm:ss 或 yyyy-MM-dd HH:mm:ss。
- 不要携带时区、毫秒、小数秒或 ISO 字母 T。
- 空白时间不会报错，但会传入 double[6] 全 0；生产任务建议显式提供。

7. 如何处理并发任务

- 若原生库没有明确线程安全承诺，应在调用方串行化 startMonitor()。
- 不要依赖共享单例中的可变回调区分并发任务。可按任务封装结果处理，或自定义 MonitorDataService。
- 如需直接 JNA 串行包装，可评估 MonitorLibrary.INSTANCES，但默认服务未使用它。

8. 如何重置 SDK

- 当前 Java API 没有初始化或关闭方法。完成一次调用后可构建新的 MonitorTask 再次调用。
- 若原生库进入不可恢复状态，应依据原生库交付说明决定是否重启进程。

附录 A：完整 Spring Boot 示例

```

package com.example.monitor;

import com.navfirst.dmonitor.lib.domains.MonitorTask;
import com.navfirst.dmonitor.lib.services.MonitorService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.nio.file.Files;
import java.nio.file.Path;

@Service
@RequiredArgsConstructor
public class DMonitorRunner {

    private final MonitorService monitorService;

    public void run() throws Exception {
        Path brdcPath = Path.of("/data/BRDM1400.rnx");
        byte[] brdc = Files.isRegularFile(brdcPath)
            ? Files.readAllBytes(brdcPath)
            : null;

        MonitorTask task = MonitorTask.builder()
            .rtMode(3)
            .timeStart("2026/05/20 00:00:00")
            .timeEnd("2026/05/20 01:00:00")
            .filterPeriod(7200)
            .processInterval(600)
            .sample(0)
            .vrs(0)
            .roverName("XPJZ01")
            .baseName("XPJZ02")
            .brdcBytes(brdc)
            .roverBytes(Files.readAllBytes(Path.of("/data/rover.rtc3")))
            .baseBytes(Files.readAllBytes(Path.of("/data/base.rtc3")))
            .outMode(1)
            .minFixedRate(0.75)
            .navSys("1,4,8,32")
            .bds(1)
            .build();

        monitorService.setHandlerData(result -> {
            if (result.getErrMsg() != null && !result.getErrMsg().isBlank()) {
                System.err.println("算法信息: " + result.getErrMsg());
            }
        });
        System.out.printf(
            "time=%s, E=%f, N=%f, U=%f, fixedRate=%f%n",
            result.getGpsTime(),
            result.getE(),
            result.getN(),
            result.getU(),
            result.getFixedRate());
    });

    System.out.println("native version=" + monitorService.getVersion());
    monitorService.startMonitor(task, "/data/license.lic");
}
}

```

附录 B：直接 JNA 调用示例

```
import com.navfirst.dmonitor.lib.library.MonitorLibrary;
import com.navfirst.dmonitor.lib.library.MonitorStreamInfo;
import com.navfirst.dmonitor.lib.utils.TimeUtils;

import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Arrays;

public class DirectCallDemo {
    public static void main(String[] args) throws Exception {
        byte[] rover = Files.readAllBytes(Path.of("/data/rover.rtc3"));
        byte[] base = Files.readAllBytes(Path.of("/data/base.rtc3"));
        byte[] brdc = Files.readAllBytes(Path.of("/data/BRDM1400.rnx"));

        MonitorStreamInfo info = MonitorStreamInfo.builder()
            .calMode(3)
            .es(TimeUtils.parseEpoch("2026/05/20 00:00:00"))
            .ee(TimeUtils.parseEpoch("2026/05/20 01:00:00"))
            .ti(0)
            .vrsMode(0)
            .navsys(45)
            .solstatic(1)
            .fixThresh(0.75)
            .rb(new double[] {0D, 0D, 0D})
            .outfile("")
            .build();

        info.setBrdc(appendZero(brdc));
        info.setRover(rover);
        info.setBase(base);

        String raw = MonitorLibrary.INSTANCE.startGMonitor(
            info,
            "/data/license.lic");
        String[] lines = raw == null
            ? new String[] { "", "" }
            : raw.split("\\R", 2);

        System.out.println("solBuf=" + lines[0]);
        System.out.println("errMsg=" + (lines.length > 1 ? lines[1] : ""));
    }

    private static byte[] appendZero(byte[] value) {
        if (value == null || value.length == 0) {
            return value;
        }
        if (value[value.length - 1] == 0) {
            return value;
        }
        return Arrays.copyOf(value, value.length + 1);
    }
}
```

附录 C：工程包结构

包 / 资源	用途
com.navfirst.dmonitor.lib.services	公共服务接口与结果回调接口。
com.navfirst.dmonitor.lib.services.impl	默认服务实现和原生结果解析。
com.navfirst.dmonitor.lib.domains	MonitorTask 输入模型与 MonitorData 输出模型。
com.navfirst.dmonitor.lib.library	JNA 接口、结构体和原生内存封装。
com.navfirst.dmonitor.lib.enums	观测数据状态与星历状态说明。
com.navfirst.dmonitor.lib.configurations	Spring Boot 自动配置。
com.navfirst.dmonitor.lib.utils	时间转换与字节字符串辅助方法。
src/main/resources/<platform>	各平台 libDMonitor 原生库。
META-INF/spring/...AutoConfiguration.imports	Spring Boot 3 自动配置入口。

附录 D：构建与验证

构建完整工程：

```
./mvnw clean test
```

只验证版本和 UUID 接口：

```
./mvnw '-Dtest=DmonitorJavaLibApplicationTests#testVersion+testUuid' test
```

文档生成前已在 macOS arm64 环境完成上述两项接口测试：2 项通过，0 失败，0 错误；原生算法版本返回 2.0.6.8。完整监测测试依赖本地 RTCM3 和 license 文件，工程测试在文件缺失时会跳过。

文档结束 | v1.0.0 | 2026-07-29